



<https://algs4.cs.princeton.edu>

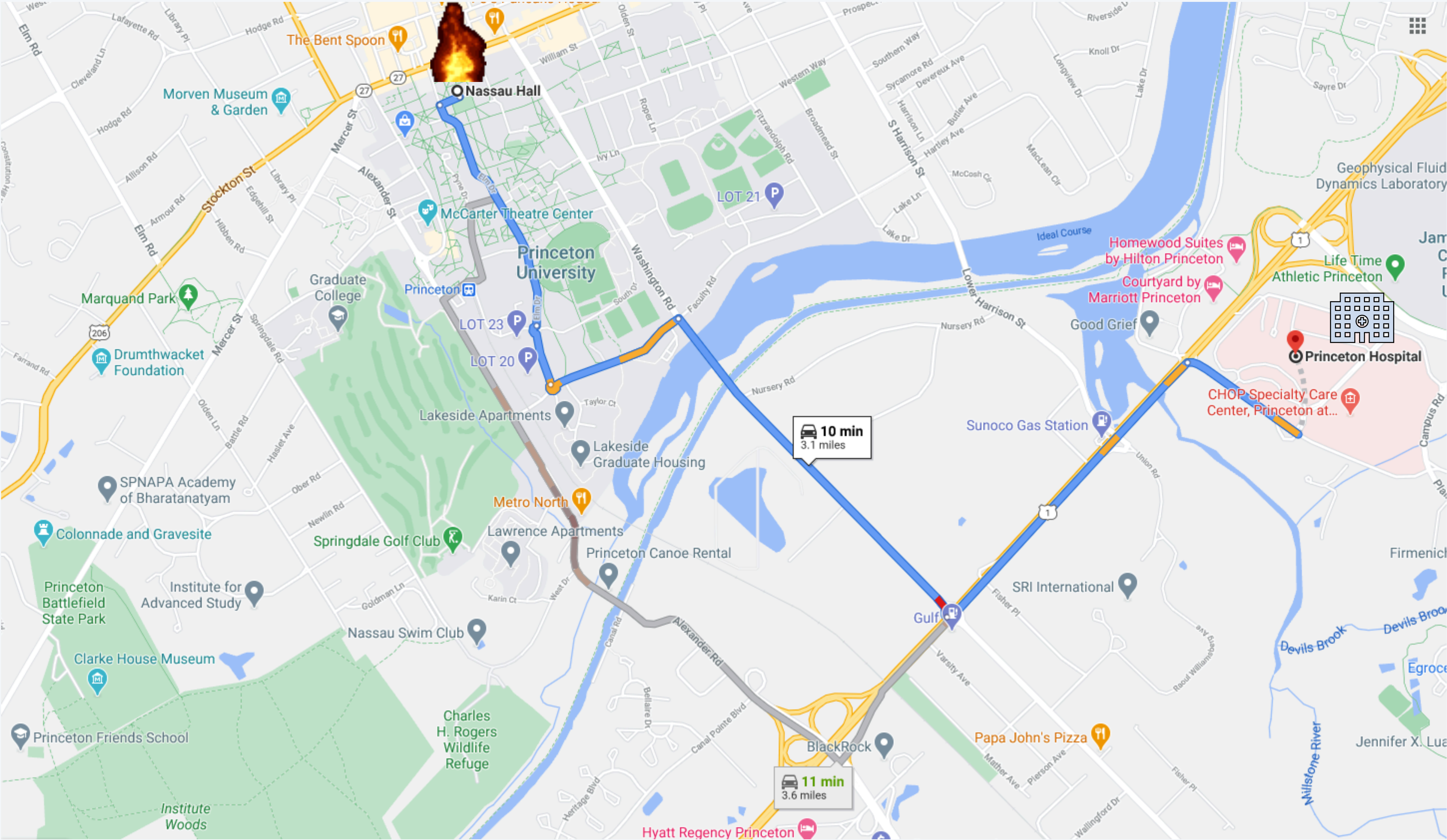
## 4.4 SHORTEST PATHS

---

- *properties*
- *APIs*
- *Bellman–Ford algorithm*
- *Dijkstra’s algorithm*



# Google maps



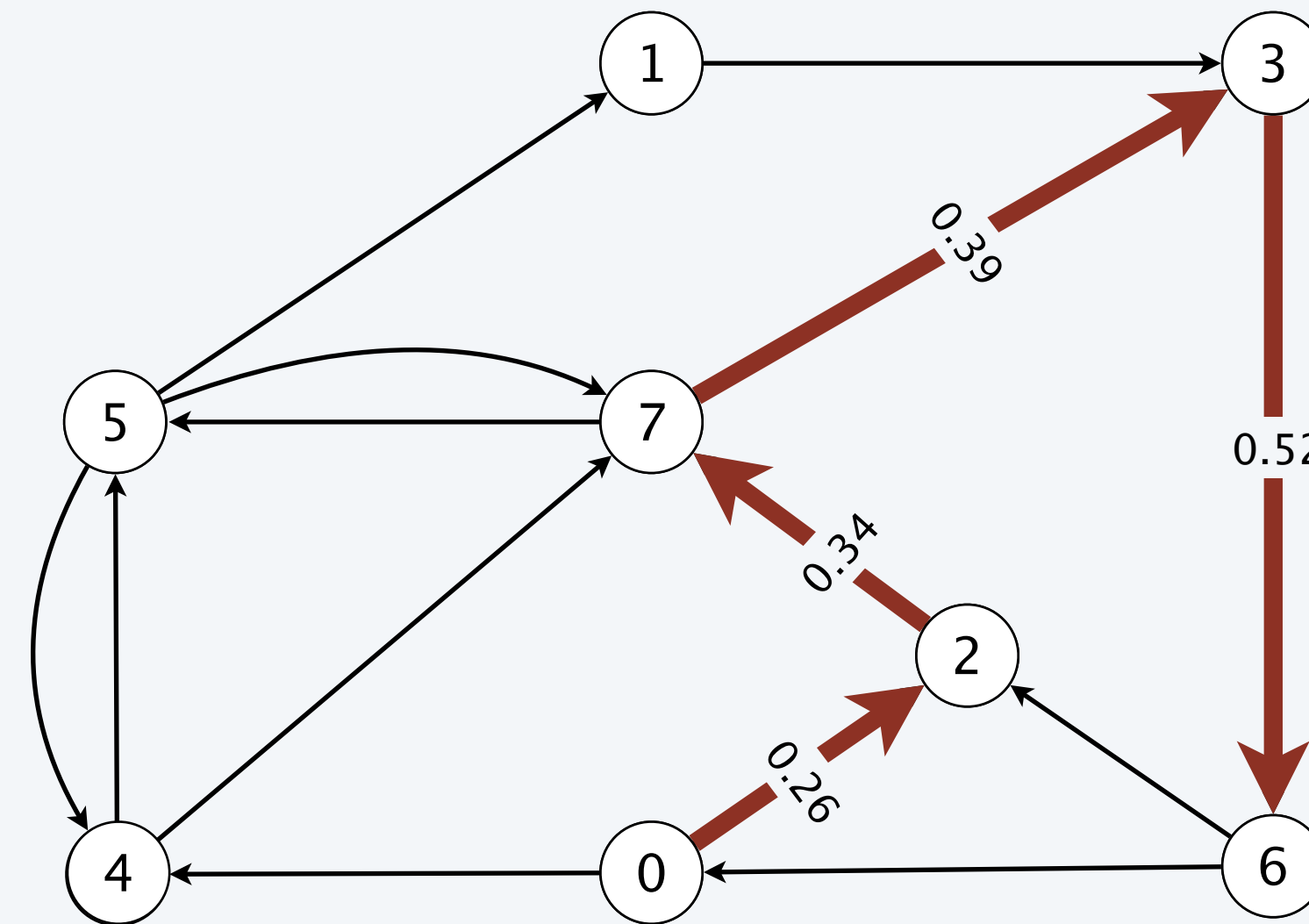


# Shortest path in an edge-weighted digraph

Given an edge-weighted digraph, find a shortest path from one vertex to another vertex.

edge-weighted digraph

|     |      |
|-----|------|
| 4→5 | 0.35 |
| 5→4 | 0.35 |
| 4→7 | 0.37 |
| 5→7 | 0.28 |
| 7→5 | 0.28 |
| 5→1 | 0.32 |
| 0→4 | 0.38 |
| 0→2 | 0.26 |
| 7→3 | 0.39 |
| 1→3 | 0.29 |
| 2→7 | 0.34 |
| 6→2 | 0.40 |
| 3→6 | 0.52 |
| 6→0 | 0.58 |
| 6→4 | 0.93 |



shortest path from 0 to 6

$0 \rightarrow 2 \rightarrow 7 \rightarrow 3 \rightarrow 6$

length of path = 1.51

$(0.26 + 0.34 + 0.39 + 0.52)$

# Shortest path applications

---

- PERT/CPM.
- Map routing.
- Seam carving.  *see Assignment 6*
- Texture mapping.
- Robot navigation.
- Typesetting in  $\text{T}_\text{E}\text{X}$ .
- Currency exchange.
- Urban traffic planning.
- Optimal pipelining of VLSI chip.
- Telemarketer operator scheduling.
- Routing of telecommunications messages.
- Network routing protocols (OSPF, BGP, RIP).
- Optimal truck routing through given traffic congestion pattern.



Reference: Network Flows: Theory, Algorithms, and Applications, R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, Prentice Hall, 1993.

# Shortest path variants

---

## Which vertices?

- Source-destination: from one vertex to another vertex.
- Single source: from one vertex to every vertex.
- Single destination: from every vertex to one vertex.
- All pairs: between all pairs of vertices.

## Restrictions on edge weights?

- Non-negative weights.  *we assume this in today's lecture (except as noted)*
- Euclidean weights.
- Arbitrary weights.

## Directed cycles?

- Prohibit.  *can derive faster algorithms (see next lecture)*
- Allow.

*implies that shortest path from  $s$  to  $v$  exists  
(and that  $E \geq V - 1$ )*

**Simplifying assumption.** Each vertex  $v$  is reachable from  $s$ .





Which shortest path variant for car GPS?

Hint: drivers make wrong turns occasionally.

- A. Source-destination: from one vertex to another vertex.
- B. Single source: from one vertex to every vertex.
- C. Single destination: from every vertex to one vertex.
- D. All pairs: between all pairs of vertices.

车载系统，属于single destination: from every vertex to one destination

因为司机中途走错路了可以马上更新路径

起点：当前车辆位置（随时变）

终点：用户设置的目的地（固定）







<https://algs4.cs.princeton.edu>

## 4.4 SHORTEST PATHS

---

- *properties*
- *APIs*
- *Bellman–Ford algorithm*
- *Dijkstra’s algorithm*

# Data structures for single-source shortest paths

**Goal.** Find a shortest path from  $s$  to every vertex.

*no repeated vertices*  
 $\Rightarrow \leq V - 1$  edges

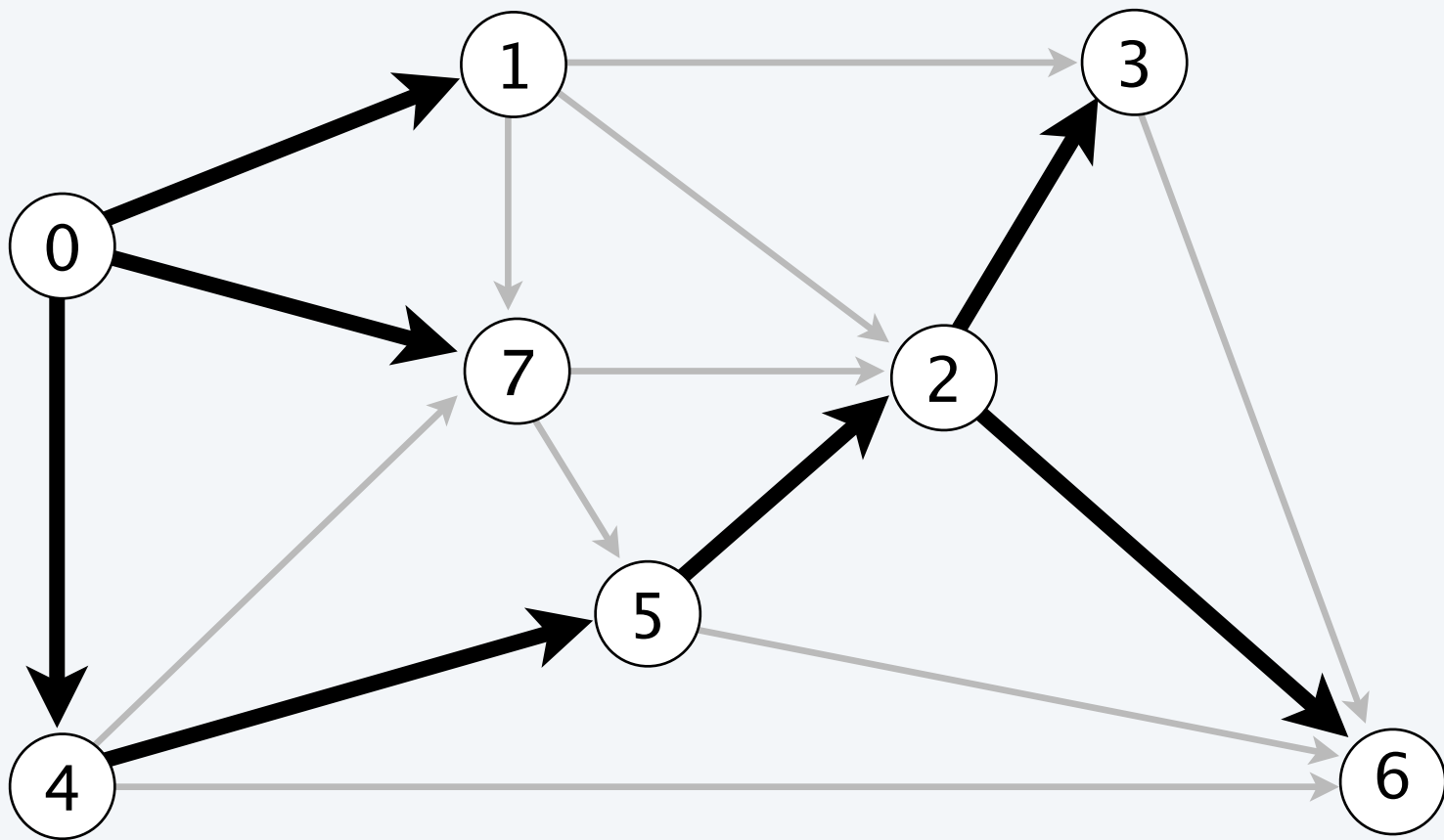
**Observation 1.** There exists a shortest path from  $s$  to  $v$  that is simple.

**Observation 2.** A **shortest-paths tree** (SPT) solution exists. Why?

结论：从一个起点  $s$  到所有节点的最短路径，一定会构成一棵「有向树」，绝对不会是普通图

**Consequence.** Can represent shortest paths with two vertex-indexed arrays:

- $\text{distTo}[v]$  is length of a shortest path from  $s$  to  $v$ .
- $\text{edgeTo}[v]$  is last edge on a shortest path from  $s$  to  $v$ .



shortest-paths tree from 0

| $v$ | $\text{distTo}[]$ | $\text{edgeTo}[]$ |
|-----|-------------------|-------------------|
| 0   | 0.0               | -                 |
| 1   | 5.0               | 0→1               |
| 2   | 14.0              | 5→2               |
| 3   | 17.0              | 2→3               |
| 4   | 9.0               | 0→4               |
| 5   | 13.0              | 4→5               |
| 6   | 25.0              | 2→6               |
| 7   | 8.0               | 0→7               |

parent-link representation



# Edge relaxation

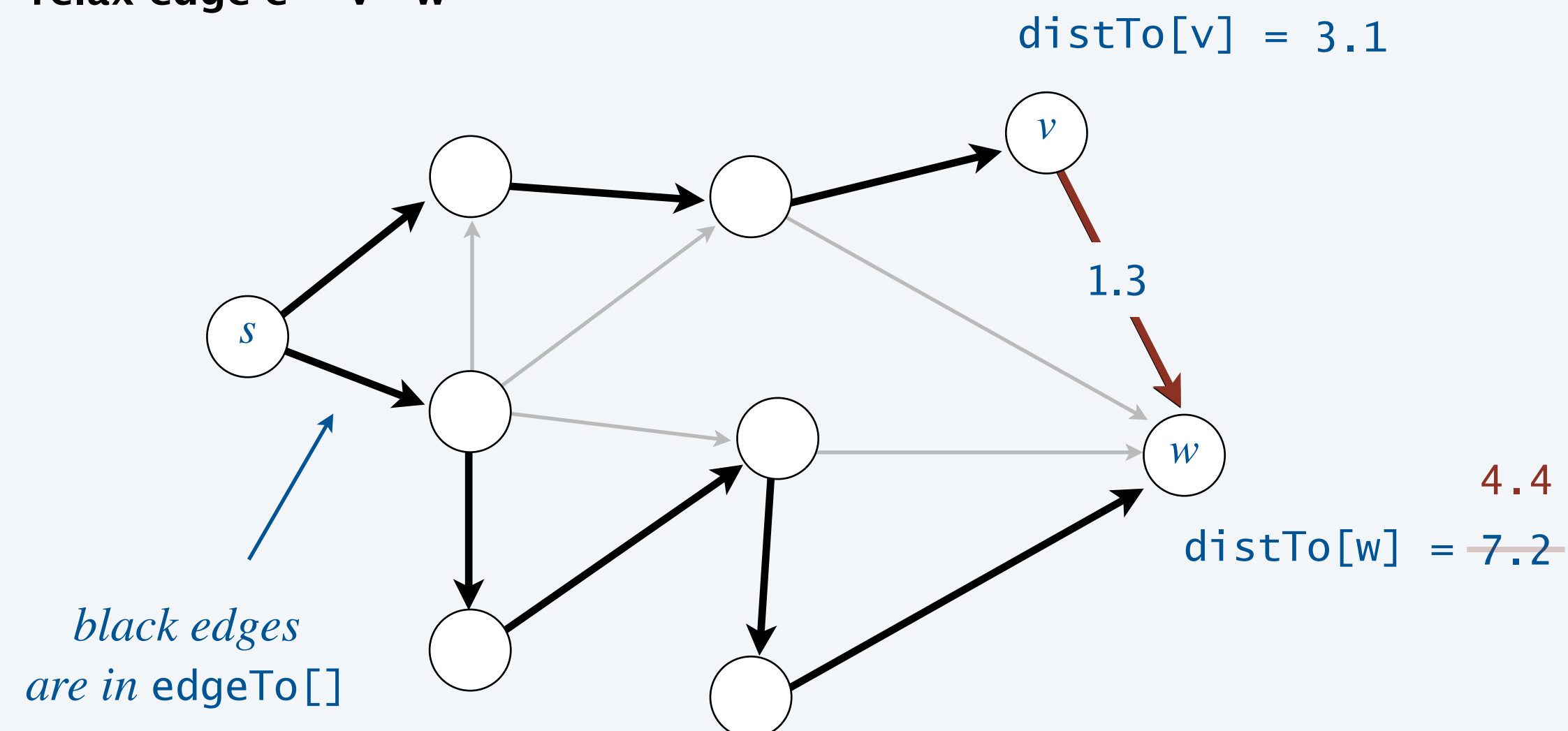
更新最短路径的过程叫做“松弛”【relaxing】  
松弛操作本质：试图通过边  $u \rightarrow v$ ，优化从起点到  $v$  的最短路径。

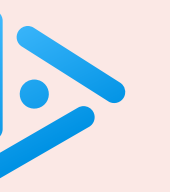
Relax edge  $e = v \rightarrow w$ .

当前已知的

- $\text{distTo}[v]$  is length of shortest **known** path from  $s$  to  $v$ .
- $\text{distTo}[w]$  is length of shortest **known** path from  $s$  to  $w$ .
- $\text{edgeTo}[w]$  is last edge on shortest **known** path from  $s$  to  $w$ .
- If  $e = v \rightarrow w$  yields shorter path from  $s$  to  $w$ , via  $v$ , update  $\text{distTo}[w]$  and  $\text{edgeTo}[w]$ .

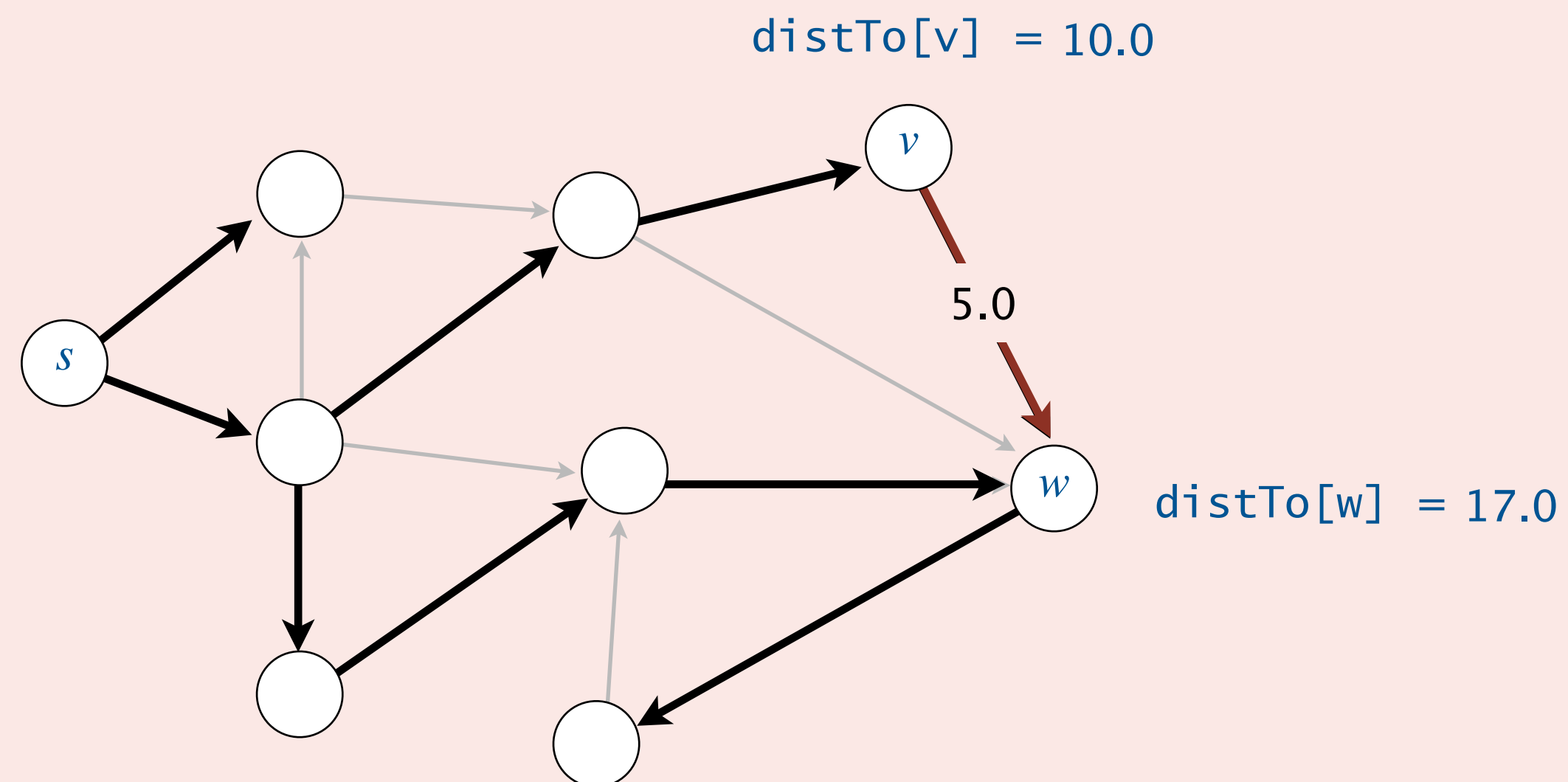
relax edge  $e = v \rightarrow w$





What are the values of  $\text{distTo}[v]$  and  $\text{distTo}[w]$  after relaxing edge  $e = v \rightarrow w$  ?

- A. 10.0 and 15.0
- B. 10.0 and 17.0
- C. 12.0 and 15.0
- D. 12.0 and 17.0



# Framework for shortest-paths algorithm

---

## Generic algorithm (to compute a SPT from $s$ )

---

For each vertex  $v$ :  $\text{distTo}[v] = \infty$ .

For each vertex  $v$ :  $\text{edgeTo}[v] = \text{null}$ .

$\text{distTo}[s] = 0$ .

Repeat until  $\text{distTo}[v]$  values converge:

- Relax any edge.
- 

**Key properties.** Throughout the generic algorithm,

- $\text{distTo}[v]$  is either infinity or the length of a (simple) path from  $s$  to  $v$ .
- $\text{distTo}[v]$  does not increase.



# Framework for shortest-paths algorithm

---

## Generic algorithm (to compute a SPT from $s$ )

---

For each vertex  $v$ :  $\text{distTo}[v] = \infty$ .

For each vertex  $v$ :  $\text{edgeTo}[v] = \text{null}$ .

$\text{distTo}[s] = 0$ .

Repeat until  $\text{distTo}[v]$  values converge:

- Relax any edge.
- 

## Efficient implementations.

- Which edge to relax next?
- How many edge relaxations needed to guarantee convergence?

Ex 1. Bellman–Ford algorithm.

Ex 2. Dijkstra's algorithm.

Ex 3. Topological sort algorithm.



<https://algs4.cs.princeton.edu>

## 4.4 SHORTEST PATHS

---

- *properties*
- *APIs*
- *Bellman–Ford algorithm*
- *Dijkstra’s algorithm*

# Weighted directed edge API

## API.

```
public class DirectedEdge
```

```
    DirectedEdge(int v, int w, double weight)    create weighted edge  $v \rightarrow w$ 
```

```
    int    from()                                vertex  $v$ 
```

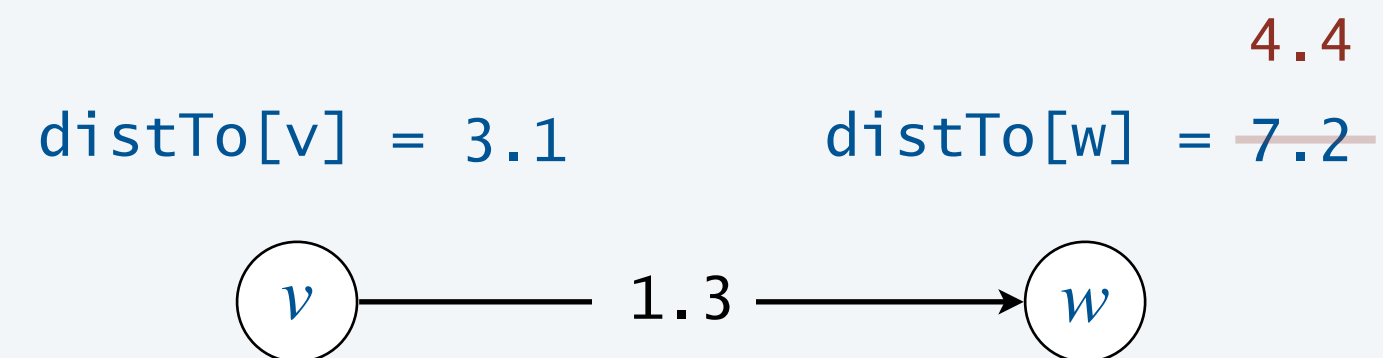
```
    int    to()                                  vertex  $w$ 
```

```
    double weight()                             weight of this edge
```

```
        :                                       :
```

Ex. Relax edge  $e = v \rightarrow w$ .

```
private void relax(DirectedEdge e) {  
    int v = e.from(), w = e.to();  
    if (distTo[w] > distTo[v] + e.weight()) {  
        distTo[w] = distTo[v] + e.weight();  
        edgeTo[w] = e;  
    }  
}
```





# Weighted directed edge: implementation in Java

---

```
public class DirectedEdge {  
    private final int v, w;  
    private final double weight;
```

```
    public DirectedEdge(int v, int w, double weight) {  
        this.v = v;  
        this.w = w;  
        this.weight = weight;  
    }
```

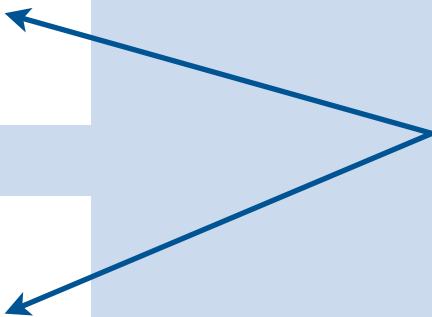
```
    public int from() {  
        return v;  
    }
```

```
    public int to() {  
        return w;  
    }
```

```
    public double weight() {  
        return weight;  
    }
```

```
}
```

*from() and to() replace  
either() and other()*



# Edge-weighted digraph API

---

API. Same as `EdgeWeightedGraph` except with `DirectedEdge` objects.

| public class EdgeWeightedDigraph |                            |                                                             |
|----------------------------------|----------------------------|-------------------------------------------------------------|
|                                  | EdgeWeightedDigraph(int V) | <i>edge-weighted digraph with V vertices (and no edges)</i> |
| void                             | addEdge(DirectedEdge e)    | <i>add weighted directed edge e</i>                         |
| Iterable<DirectedEdge>           | adj(int v)                 | <i>edges incident from v</i>                                |
| int                              | V()                        | <i>number of vertices</i>                                   |
|                                  | ⋮                          | ⋮                                                           |

# Edge-weighted digraph: adjacency-lists implementation in Java

---

**Implementation.** Almost identical to EdgeWeightedGraph.

```
public class EdgeWeightedDigraph {  
    private final int V;  
    private final Bag<DirectedEdge>[] adj;
```

```
    public EdgeWeightedDigraph(int V) {  
        this.V = V;  
        adj = (Bag<Edge>[]) new Bag[V];  
        for (int v = 0; v < V; v++)  
            adj[v] = new Bag<>();  
    }
```

```
    public void addEdge(DirectedEdge e) {  
        int v = e.from();  
        adj[v].add(e);  
    }
```

```
    public Iterable<DirectedEdge> adj(int v) {  
        return adj[v];  
    }
```

```
}
```

← *add edge  $e = v \rightarrow w$  only  
to  $v$ 's adjacency list*



# Single-source shortest paths API

---

**Goal.** Find the shortest path from  $s$  to every other vertex.

```
public class SP
```

---

|  |                                  |                                           |
|--|----------------------------------|-------------------------------------------|
|  | SP(EdgeWeightedDigraph G, int s) | <i>shortest paths from s in digraph G</i> |
|--|----------------------------------|-------------------------------------------|

|        |               |                                            |
|--------|---------------|--------------------------------------------|
| double | distTo(int v) | <i>length of shortest path from s to v</i> |
|--------|---------------|--------------------------------------------|

|                         |               |                                  |
|-------------------------|---------------|----------------------------------|
| Iterable <DirectedEdge> | pathTo(int v) | <i>shortest path from s to v</i> |
|-------------------------|---------------|----------------------------------|

|         |                  |                                      |
|---------|------------------|--------------------------------------|
| boolean | hasPathTo(int v) | <i>is there a path from s to v ?</i> |
|---------|------------------|--------------------------------------|





<https://algs4.cs.princeton.edu>

## 4.4 SHORTEST PATHS

---

- *properties*
- *APIs*
- *Bellman–Ford algorithm*
- *Dijkstra’s algorithm*

# Bellman-Ford algorithm

## Bellman-Ford algorithm

For each vertex  $v$ :  $\text{distTo}[v] = \infty$ .

For each vertex  $v$ :  $\text{edgeTo}[v] = \text{null}$ .

$\text{distTo}[s] = 0$ .

Repeat  $V-1$  times:

- Relax each edge.

```
for (int i = 1; i < G.V(); i++)
    for (int v = 0; v < G.V(); v++)
        for (DirectedEdge e : G.adj(v))
            relax(e);
```

```
private void relax(DirectedEdge e) {
    int v = e.from(), w = e.to();
    if (distTo[w] > distTo[v] + e.weight()) {
        distTo[w] = distTo[v] + e.weight();
        edgeTo[w] = e;
    }
}
```

// 外层循环：执行  $V-1$  轮 ( $i$  从 1 到  $G.V()-1$ ，共  $V-1$  次)，  
// 第  $k$  轮循环 (从 1 开始) 后，可以保证：所有最多经过  $k$  条边的最短路径，都已经被算出来。

```
for (int i = 1; i < G.V(); i++)
    // 中层循环：遍历每个顶点 v
    for (int v = 0; v < G.V(); v++)
        // 内层循环：遍历顶点 v 的所有邻接边 e (即对每条边执行松弛)
        for (DirectedEdge e : G.adj(v))
            relax(e); // 执行松弛操作
```

← *pass i (relax each edge once)*

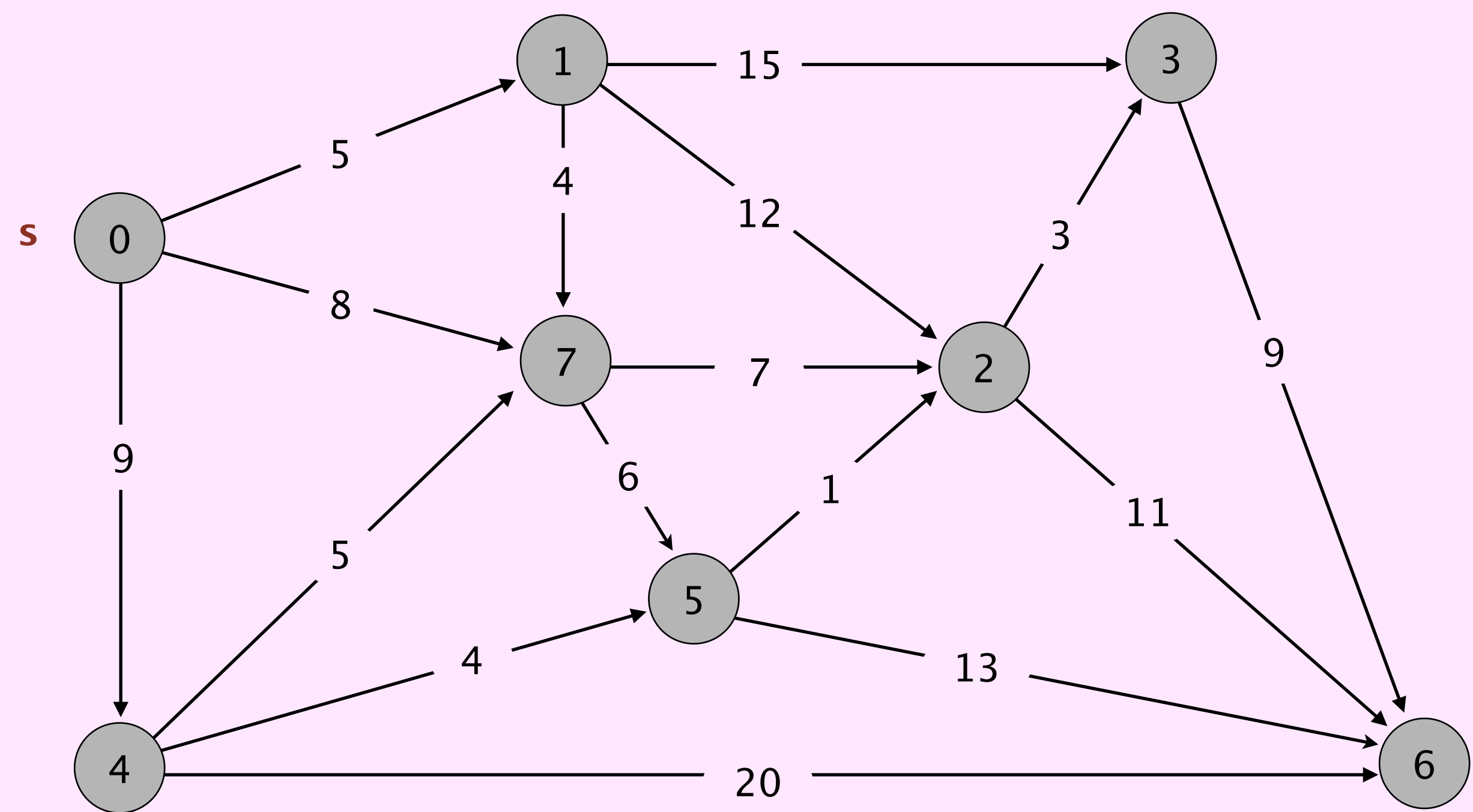
*number of calls to relax() in pass i =*  
*outdegree(0) + outdegree(1) + outdegree(2) + ... = E*

**Running time.** Algorithm takes  $\Theta(E V)$  time and uses  $\Theta(V)$  extra space.

# Bellman-Ford algorithm demo



Repeat  $V - 1$  times: relax all  $E$  edges.



an edge-weighted digraph

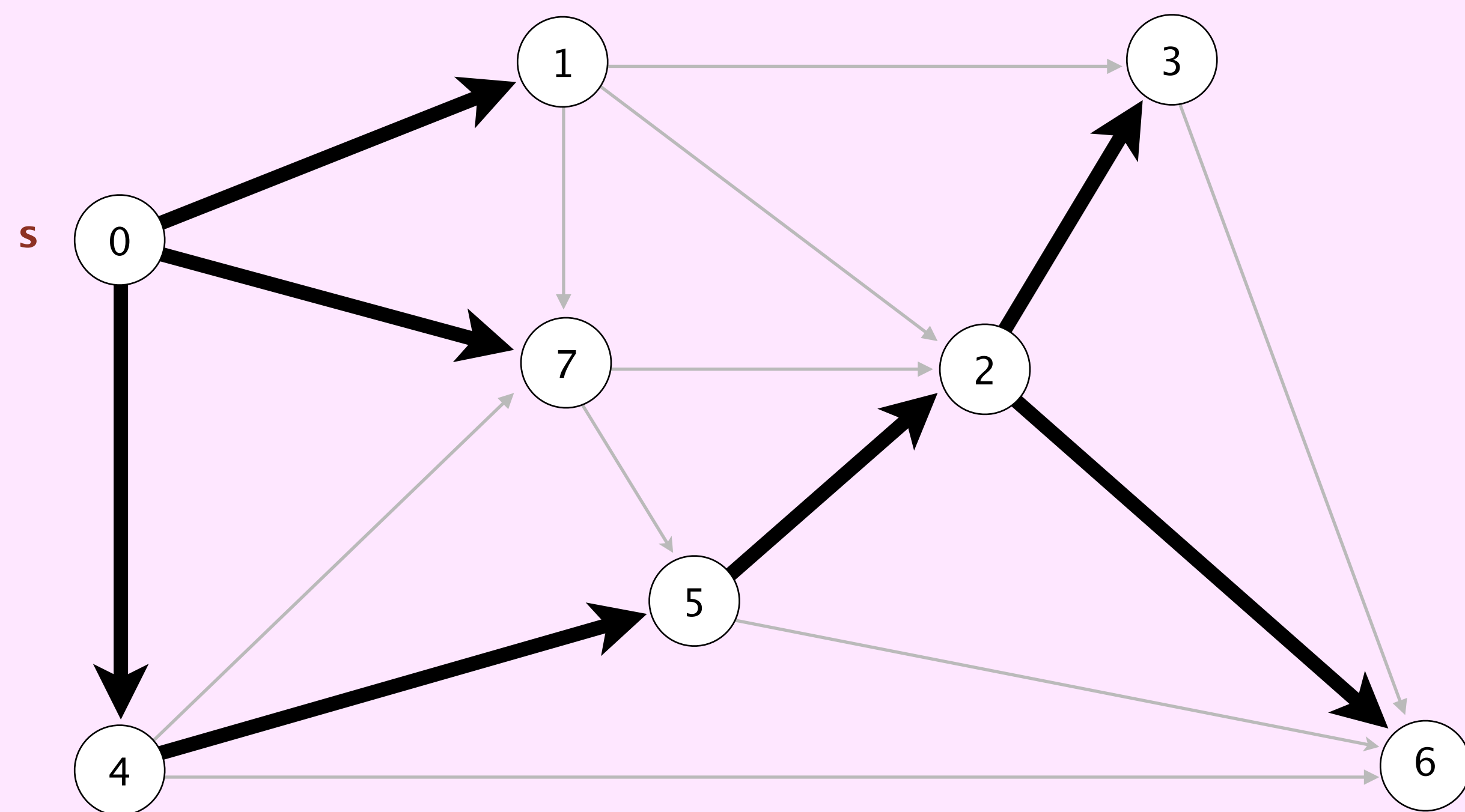
|     |      |
|-----|------|
| 0→1 | 5.0  |
| 0→4 | 9.0  |
| 0→7 | 8.0  |
| 1→2 | 12.0 |
| 1→3 | 15.0 |
| 1→7 | 4.0  |
| 2→3 | 3.0  |
| 2→6 | 11.0 |
| 3→6 | 9.0  |
| 4→5 | 4.0  |
| 4→6 | 20.0 |
| 4→7 | 5.0  |
| 5→2 | 1.0  |
| 5→6 | 13.0 |
| 7→5 | 6.0  |
| 7→2 | 7.0  |



# Bellman-Ford algorithm demo



Repeat  $V - 1$  times: relax all  $E$  edges.



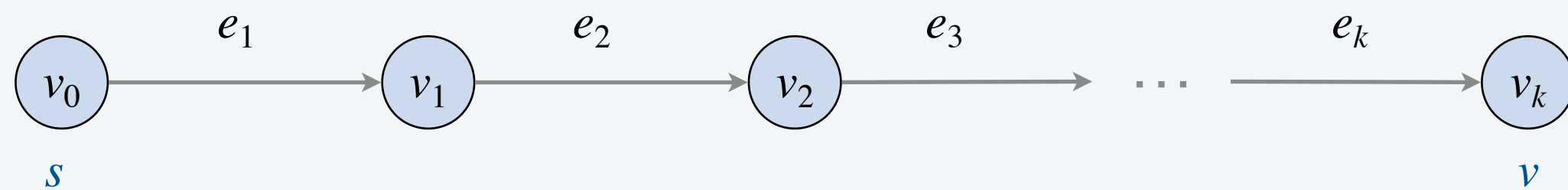
| v | distTo[] | edgeTo[] |
|---|----------|----------|
| 0 | 0.0      | -        |
| 1 | 5.0      | 0→1      |
| 2 | 14.0     | 5→2      |
| 3 | 17.0     | 2→3      |
| 4 | 9.0      | 0→4      |
| 5 | 13.0     | 4→5      |
| 6 | 25.0     | 2→6      |
| 7 | 8.0      | 0→7      |

shortest-paths tree from vertex s

# Bellman–Ford algorithm: correctness proof

**Proposition.** Let  $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v$  be any path from  $s$  to  $v$  containing  $k$  edges.

Then, after pass  $k$ ,  $\text{distTo}[v_k] \leq \text{weight}(e_1) + \text{weight}(e_2) + \dots + \text{weight}(e_k)$ .



**Pf.** [ by induction on number of passes  $i$  ]

- Base case: initially,  $0 = \text{distTo}[v_0] \leq 0$ .
- Inductive hypothesis: after pass  $i$ ,  $\text{distTo}[v_i] \leq \text{weight}(e_1) + \text{weight}(e_2) + \dots + \text{weight}(e_i)$ .
- This inequality continues to hold because  $\text{distTo}[v_i]$  cannot increase.
- Immediately after relaxing edge  $e_{i+1}$  in pass  $i+1$ , we have

$$\begin{aligned} \text{distTo}[v_{i+1}] &\leq \text{distTo}[v_i] + \text{weight}(e_{i+1}) \quad \leftarrow \text{edge relaxation} \\ &\leq \text{weight}(e_1) + \text{weight}(e_2) + \dots + \text{weight}(e_i) + \text{weight}(e_{i+1}). \quad \leftarrow \text{inductive hypothesis} \end{aligned}$$

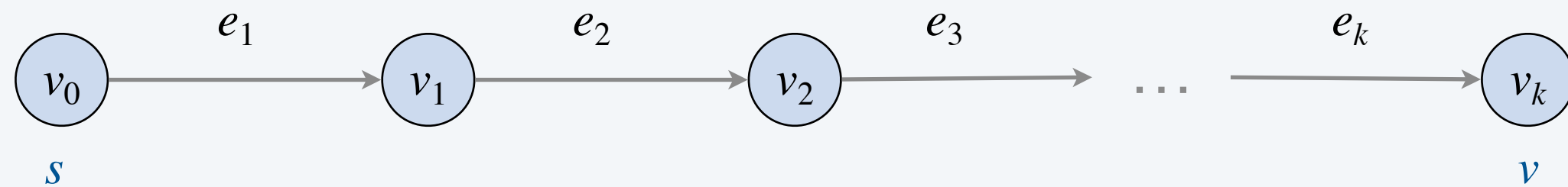
- This inequality continues to hold because  $\text{distTo}[v_{i+1}]$  cannot increase. ■



# Bellman–Ford algorithm: correctness proof

---

**Proposition.** Let  $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v$  be any path from  $s$  to  $v$  containing  $k$  edges. Then, after pass  $k$ ,  $\text{distTo}[v_k] \leq \text{weight}(e_1) + \text{weight}(e_2) + \dots + \text{weight}(e_k)$ .



**Corollary.** Bellman–Ford computes shortest path distances.

**Pf.** [ apply Proposition to a shortest path from  $s$  to  $v$  ]

- There exists a **simple shortest path  $P^*$  from  $s$  to  $v$** ; it contains  $k \leq V - 1$  edges.
- The Proposition implies that, after at most  $V - 1$  passes,  $\text{distTo}[v] \leq \text{length}(P^*)$ .
- Since  $\text{distTo}[v]$  is the length of some path from  $s$  to  $v$ ,  $\text{distTo}[v] = \text{length}(P^*)$ . ■

# Bellman-Ford algorithm: practical improvement

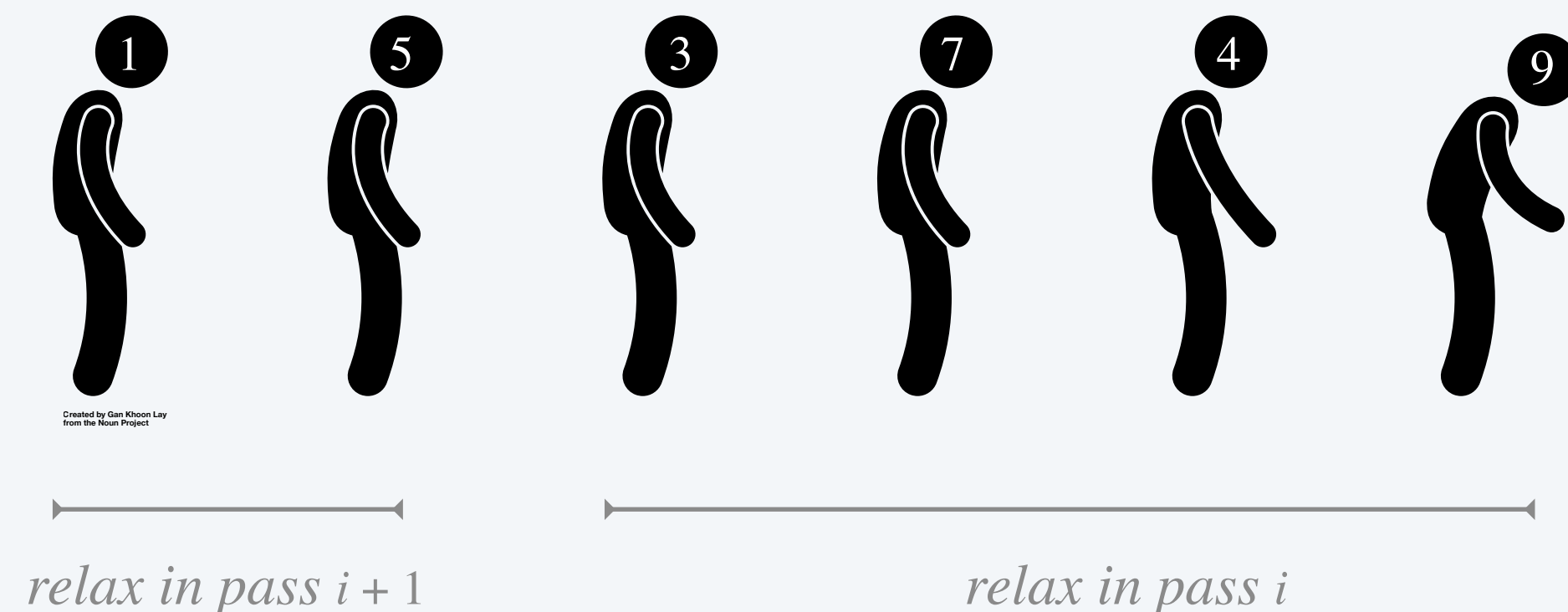
**Observation.** If `distTo[v]` does not change during pass  $i$ ,  
not necessary to relax any edges incident from  $v$  in pass  $i + 1$ .

如果节点  $v$  的最短距离 `distTo[v]` 在第  $i$  轮没有变化，  
那么第  $i+1$  轮完全不需要松弛  $v$  的任何出边。

队列实现逻辑：队列里放的是：distTo 值发生变化、还没被重新 relax 的顶点

## Queue-based implementation of Bellman-Ford.

- Perform **vertex** relaxations.  $\longleftarrow$  *relax vertex  $v$  = relax all edges incident from  $v$*
- Maintain **queue** of vertices whose `distTo[]` values changed since it was last relaxed.



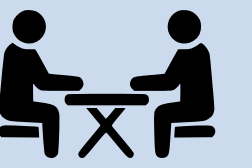
*must ensure each vertex is on queue at most once  
(or exponential blowup!)*

**relax vertex  $v$**

执行流程：  
初始将源点入队  
取出队首节点  $u$ ，松弛其所有出边  
若邻接节点  $v$  的距离被更新，且  $v$  不在队列中，则将  $v$  入队  
重复直到队列为空（无节点可更新）

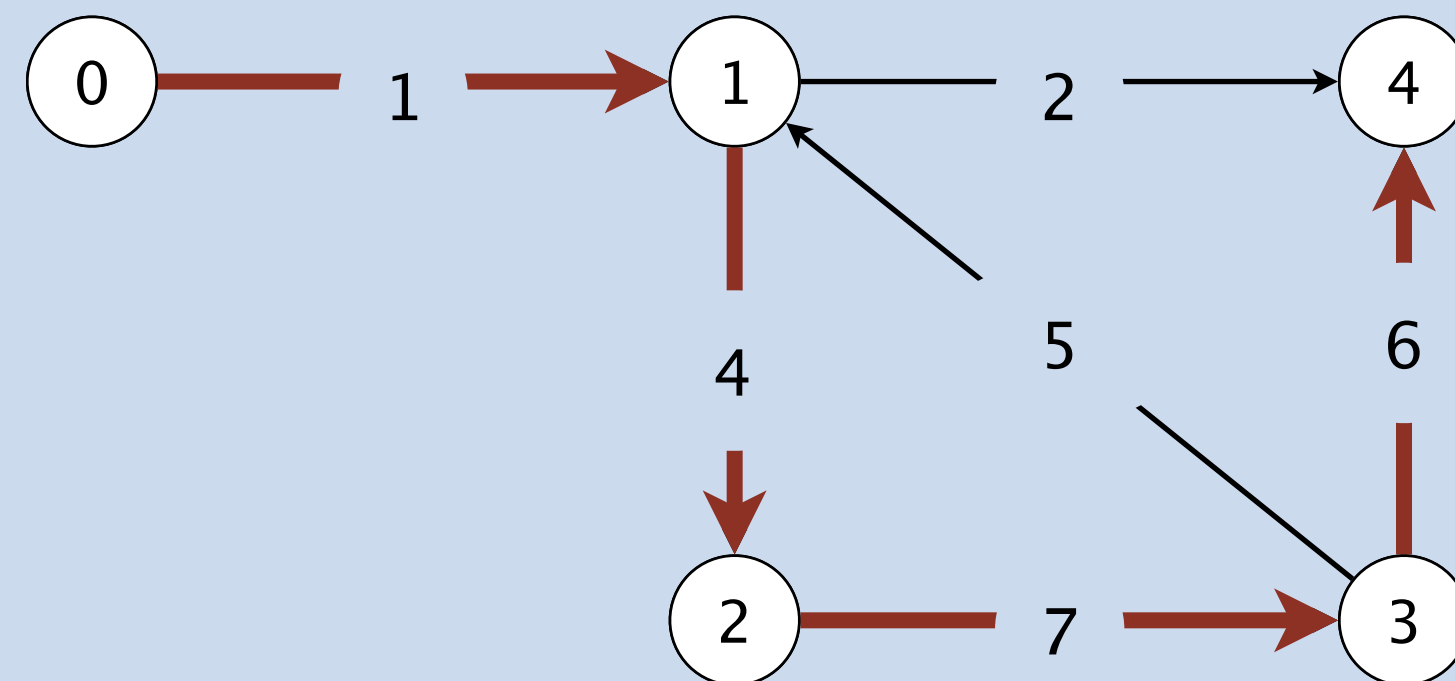
## Impact. 开销上的影响

- In the worst case, the running time is still  $\Theta(E V)$ .
- But much faster in practice on typical inputs.



**Problem.** Given a digraph  $G$  with positive edge weights and vertex  $s$ , find a **longest simple path** from  $s$  to every other vertex.

**Goal.** Design an algorithm that takes  $\Theta(E V)$  time in the worst case.



longest simple path from 0 to 4

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

length of path = 18

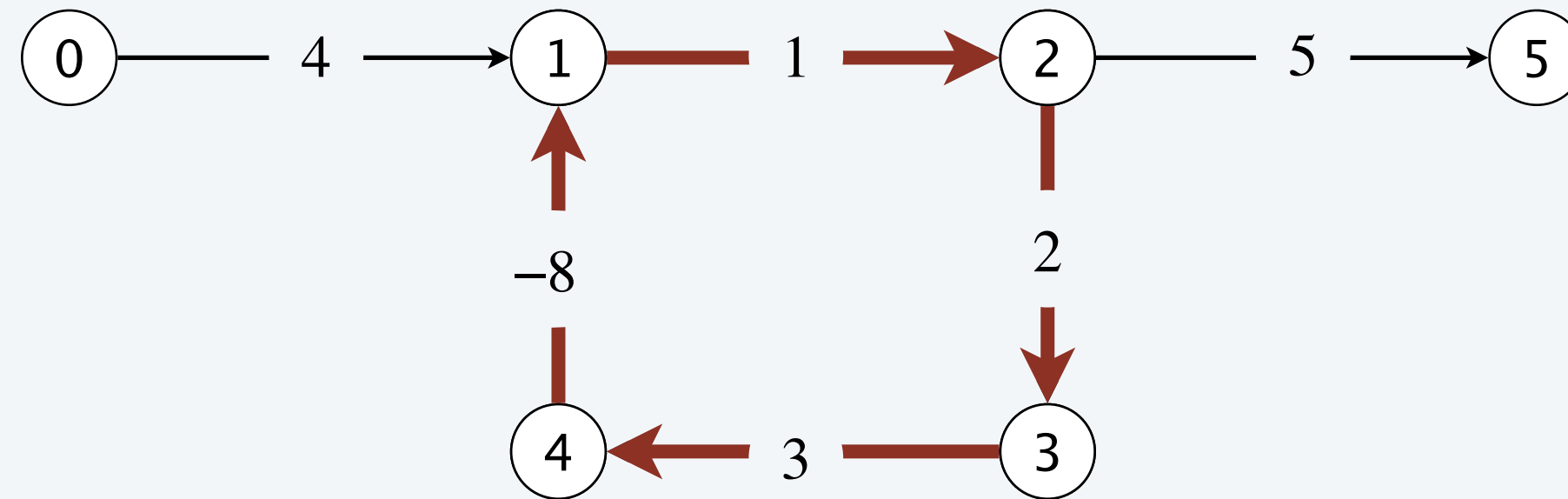
$(1 + 4 + 7 + 6)$



# Bellman–Ford algorithm: negative weights

**Remark.** The Bellman–Ford algorithm works even if some weights are negative, provided there are no **negative cycles**.

**Negative cycle.** A directed cycle whose length is negative.



**negative cycle**  
(length =  $1 + 2 + 3 + -8 = -2 < 0$ )

**Negative cycles and shortest paths.** Length of path can be made arbitrarily negative by using negative cycle.

$0 \rightarrow 1 \rightarrow \underline{2 \rightarrow 3 \rightarrow 4 \rightarrow 1} \rightarrow \dots \rightarrow \underline{2 \rightarrow 3 \rightarrow 4 \rightarrow 1} \rightarrow 2 \rightarrow 5$



<https://algs4.cs.princeton.edu>

## 4.4 SHORTEST PATHS

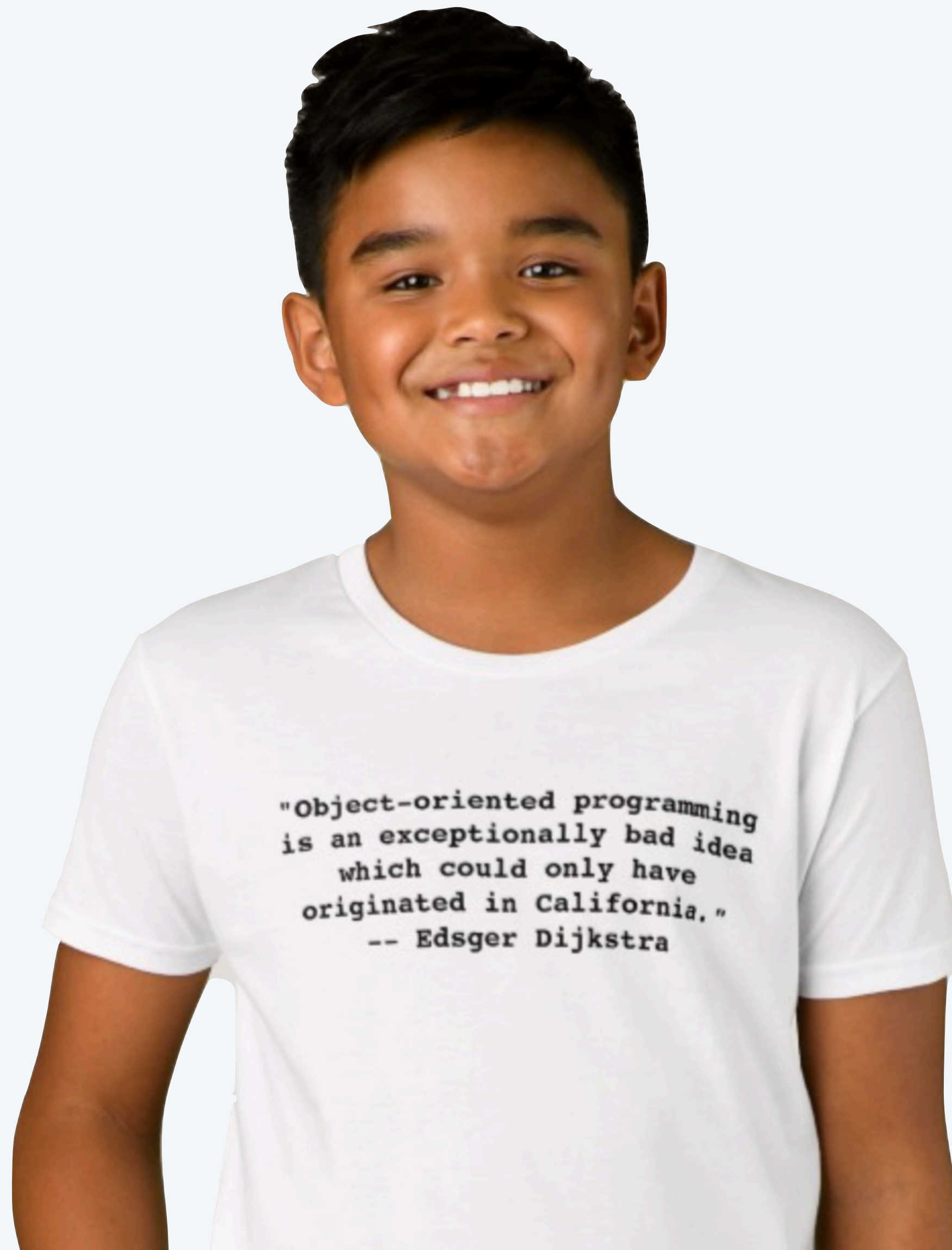
---

- *properties*
- *APIs*
- *Bellman–Ford algorithm*
- *Dijkstra’s algorithm*



## Edsger W. Dijkstra: select quote

---



# Dijkstra's algorithm

---

## Dijkstra's algorithm

---

For each vertex  $v$ :  $\text{distTo}[v] = \infty$ .

For each vertex  $v$ :  $\text{edgeTo}[v] = \text{null}$ .

$T = \emptyset$ .

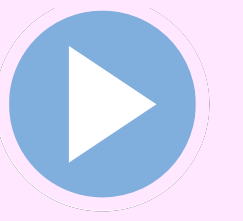
$\text{distTo}[s] = 0$ .

Repeat until all vertices are marked:

- Select unmarked vertex  $v$  with the smallest  $\text{distTo}[]$  value.
  - Mark  $v$ .
  - Relax each edge incident from  $v$ . 松弛：如果从原点经过 $v$ 到 $w$ 距离更短，就更新 $w$ 的 $\text{distTo}$ 和 $\text{edgeTo}$
- 

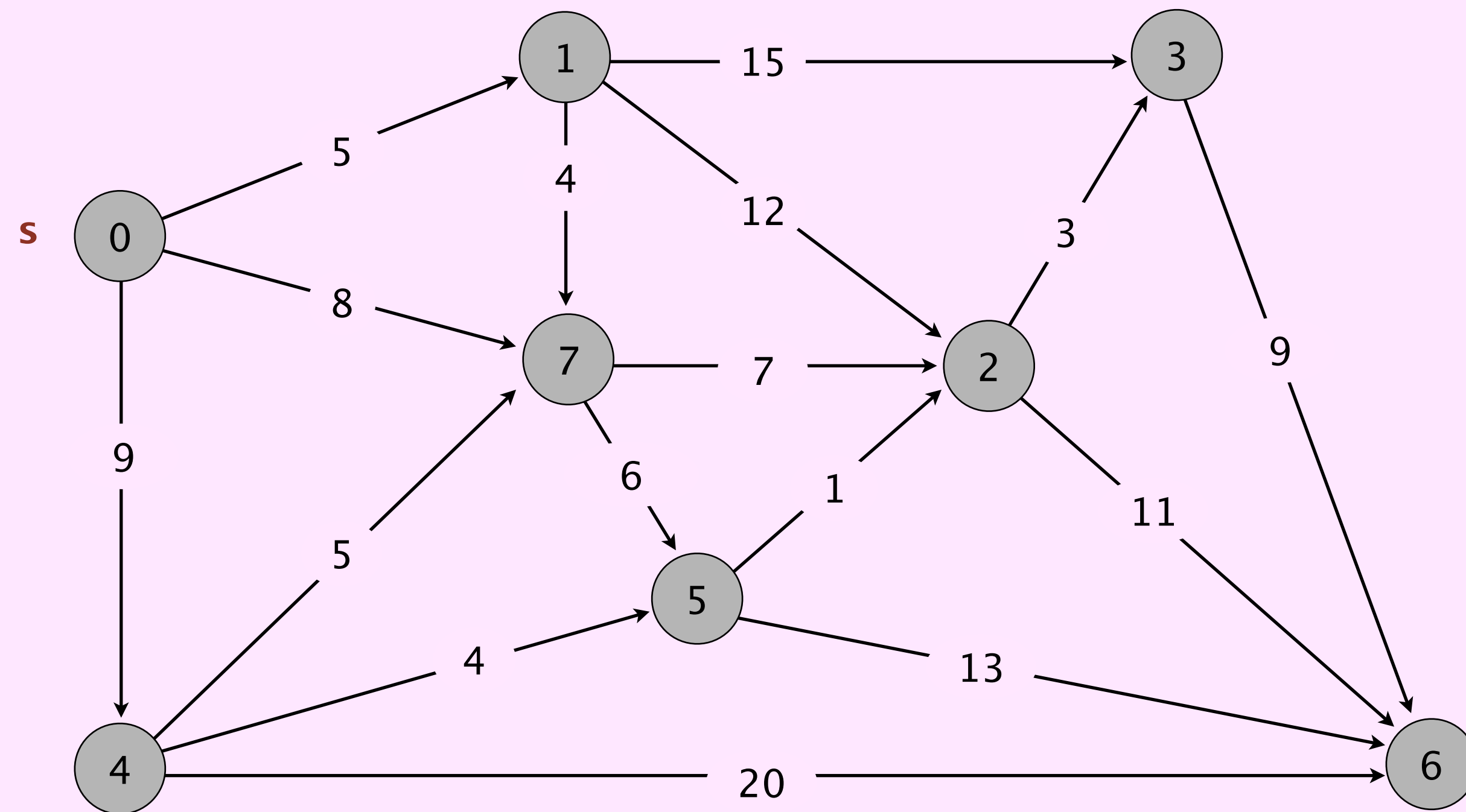
Key difference with Bellman-Ford. Each edge gets relaxed exactly once!

# Dijkstra's algorithm demo



Repeat until all vertices are marked:

- Select unmarked vertex  $v$  with the smallest `distTo[]` value.
- Mark  $v$  and relax all edges incident from  $v$ .



an edge-weighted digraph

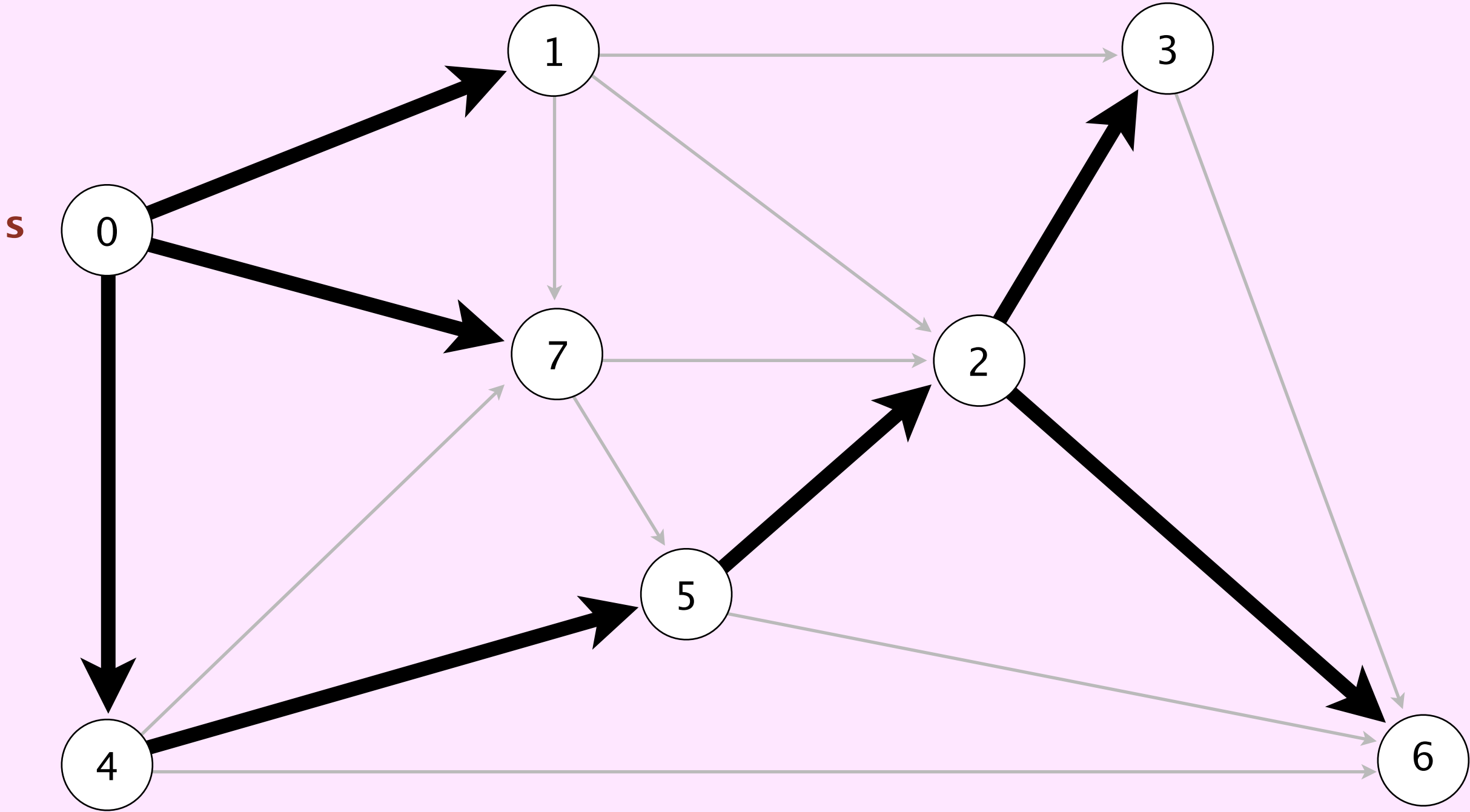
|     |      |
|-----|------|
| 0→1 | 5.0  |
| 0→4 | 9.0  |
| 0→7 | 8.0  |
| 1→2 | 12.0 |
| 1→3 | 15.0 |
| 1→7 | 4.0  |
| 2→3 | 3.0  |
| 2→6 | 11.0 |
| 3→6 | 9.0  |
| 4→5 | 4.0  |
| 4→6 | 20.0 |
| 4→7 | 5.0  |
| 5→2 | 1.0  |
| 5→6 | 13.0 |
| 7→5 | 6.0  |
| 7→2 | 7.0  |





Repeat until all vertices are marked:

- Select unmarked vertex  $v$  with the smallest `distTo[]` value.
- Mark  $v$  and relax all edges incident from  $v$ . 从 $v$ 出发的边



| v | distTo[] | edgeTo[] |
|---|----------|----------|
| 0 | 0.0      | -        |
| 1 | 5.0      | 0→1      |
| 2 | 14.0     | 5→2      |
| 3 | 17.0     | 2→3      |
| 4 | 9.0      | 0→4      |
| 5 | 13.0     | 4→5      |
| 6 | 25.0     | 2→6      |
| 7 | 8.0      | 0→7      |

shortest-paths tree from vertex s

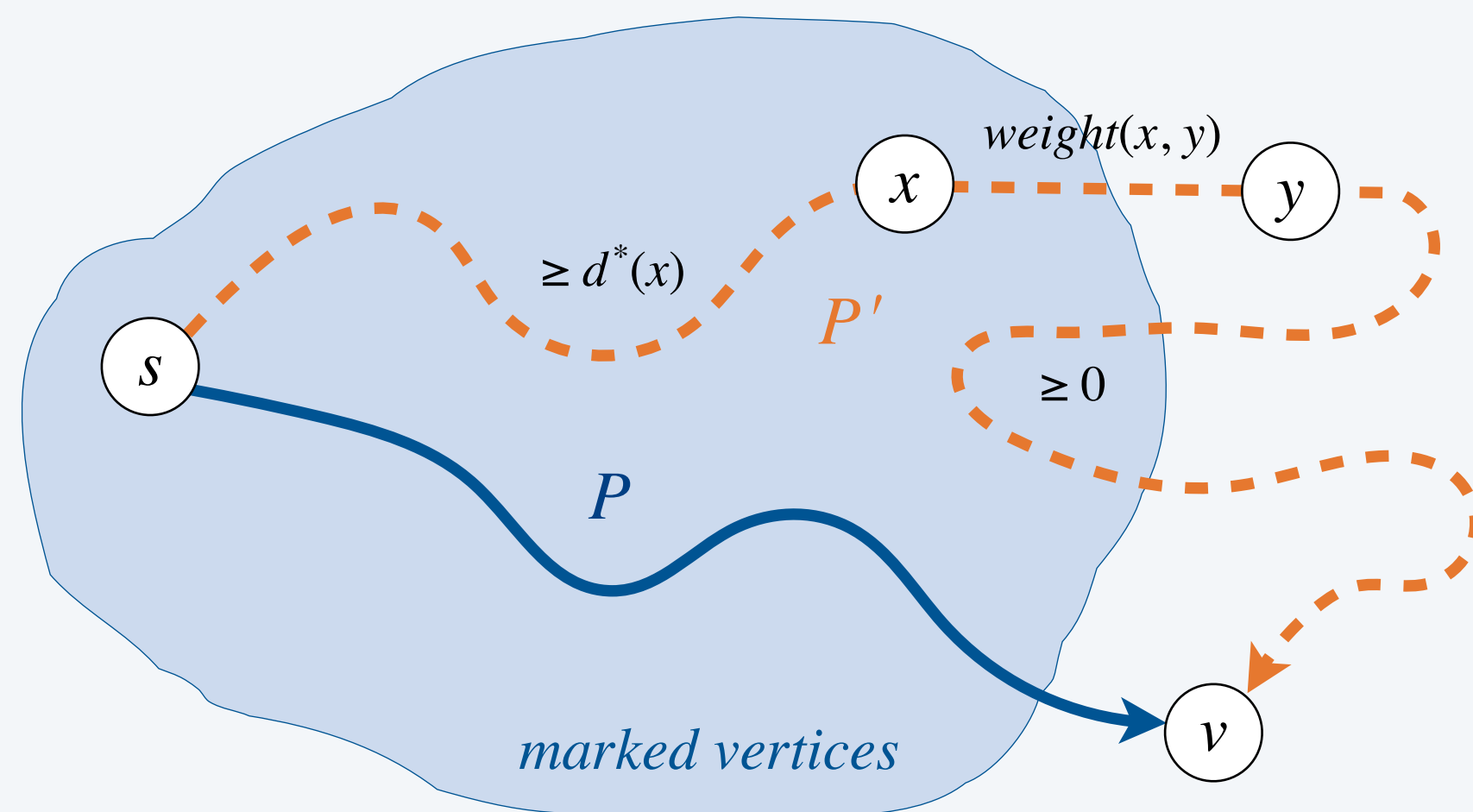
# Dijkstra's algorithm: correctness proof

**Invariant.** For each marked vertex  $v$ :  $\text{distTo}[v] = d^*(v)$ .

归纳法

**Pf.** [ by induction on number of marked vertices ]

- Let  $v$  be next vertex marked.
- Let  $P$  be the path from  $s$  to  $v$  of length  $\text{distTo}[v]$ .
- Consider any other path  $P'$  from  $s$  to  $v$ .
- Let  $x \rightarrow y$  be first edge in  $P'$  with  $x$  marked and  $y$  unmarked.
- $P'$  is already as long as  $P$  by the time it reaches  $y$ :



*length of shortest path from  $s$  to  $v$*

$\text{length}(P) = \text{distTo}[v]$

*by construction*

*Dijkstra chose  $v$  instead of  $y$*   $\rightarrow \leq \text{distTo}[y]$

*vertex  $x$  is marked (so it was relaxed)*  $\rightarrow \leq \text{distTo}[x] + \text{weight}(x, y)$

*induction*  $\rightarrow = d^*(x) + \text{weight}(x, y)$

*$P'$  is a path from  $s$  to  $x$ , followed by edge  $x \rightarrow y$ , followed by non-negative edges*  $\rightarrow \leq \text{length}(P') \quad \blacksquare$



# Dijkstra's algorithm: correctness proof

---

**Invariant.** For each marked vertex  $v$ :  $\text{distTo}[v] = d^*(v)$ .

*length of shortest path from  $s$  to  $v$*

corollary: 推论

**Corollary 1.** Dijkstra's algorithm computes shortest path distances.

**Corollary 2.** Dijkstra's algorithm relaxes vertices in increasing order of distance from  $s$ .

*generalizes both  
level-order traversal in a tree  
and breadth-first search in a graph*

迪杰斯特拉算法是BFS的一种推广  
当所有edge的权值变成1的时候，就退化成了BFS

# Dijkstra's algorithm: Java implementation

```
public class DijkstraSP {
    private DirectedEdge[] edgeTo;
    private double[] distTo;
    private IndexMinPQ<Double> pq;

    public DijkstraSP(EdgeWeightedDigraph G, int s) {
        edgeTo = new DirectedEdge[G.V()];
        distTo = new double[G.V()];
        pq = new IndexMinPQ<Double>(G.V());

        for (int v = 0; v < G.V(); v++)
            distTo[v] = Double.POSITIVE_INFINITY;
        distTo[s] = 0.0;

        pq.insert(s, 0.0);
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            for (DirectedEdge e : G.adj(v))
                relax(e);
        }
    }
}
```

*PQ that supports  
decreasing the key  
(stay tuned)*

*PQ contains the  
unmarked vertices  
with finite distTo[] values*

*relax vertices in increasing order  
of distance from s*

在relax的时候, 更新 distTo[w] 和 edgeTo[w]; 然后如果 w 已经在 pq 中, 就执行 decreaseKey, 如果 w 不在 pq 中, 就把它插入 pq。

# Dijkstra's algorithm: Java implementation

---

When relaxing an edge, also update PQ:

- Found first path from  $s$  to  $w$ : add  $w$  to PQ.
- Found better path from  $s$  to  $w$ : decrease key of  $w$  in PQ.

```
private void relax(DirectedEdge e) {  
    int v = e.from(), w = e.to();  
    if (distTo[w] > distTo[v] + e.weight()) {  
        distTo[w] = distTo[v] + e.weight();  
        edgeTo[w] = e;  
  
        if (!pq.contains(w)) pq.insert(w, distTo[w]);  
        else  
            pq.decreaseKey(w, distTo[w]);  
        找到w, 然后swim  
  
    }  
}
```

← *update PQ*

Q. How to implement DECREASE-KEY operation in a priority queue?



# Indexed priority queue (Section 2.4)

---

Associate an index between 0 and  $n - 1$  with each key in a priority queue.

- Insert a key associated with a given index.
- Delete a minimum key and return associated index.
- **Decrease the key** associated with a given index.

  
*for Dijkstra's algorithm:*  
 $n = V,$   
 $index = vertex,$   
 $key = distance\ from\ s$

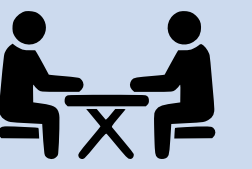
类IndexMinPQ对应的方法

```
public class IndexMinPQ<Key extends Comparable<Key>>
```

---

|                                  |                                                   |
|----------------------------------|---------------------------------------------------|
| IndexMinPQ(int n)                | <i>create PQ with indices 0, 1, ... , n - 1</i>   |
| void insert(int i, Key key)      | <i>associate key with index i</i>                 |
| int delMin()                     | <i>remove min key and return associated index</i> |
| void decreaseKey(int i, Key key) | <i>decrease the key associated with index i</i>   |
| boolean isEmpty()                | <i>is the priority queue empty ?</i>              |
| ⋮                                | ⋮                                                 |

# Decrease-Key in a Binary Heap

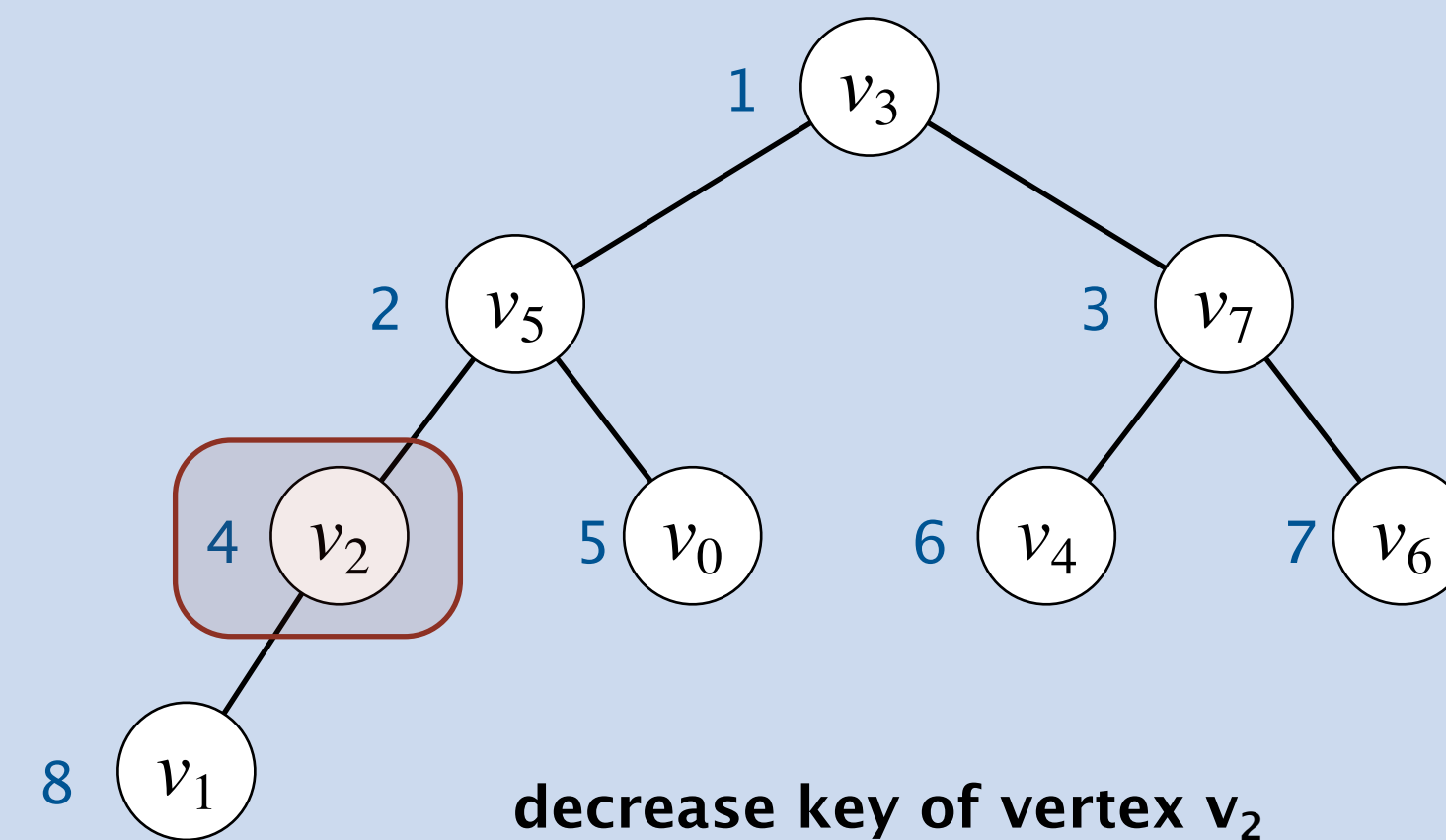


**Goal.** Implement DECREASE-KEY operation in a binary heap.

实际实现结构

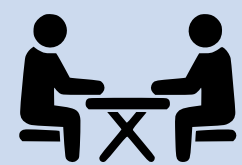
|      | 0 | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     |
|------|---|-------|-------|-------|-------|-------|-------|-------|-------|
| pq[] | — | $v_3$ | $v_5$ | $v_7$ | $v_2$ | $v_0$ | $v_4$ | $v_6$ | $v_1$ |

逻辑结构





# Decrease-Key in a Binary Heap



**Goal.** Implement DECREASE-KEY operation in a binary heap.

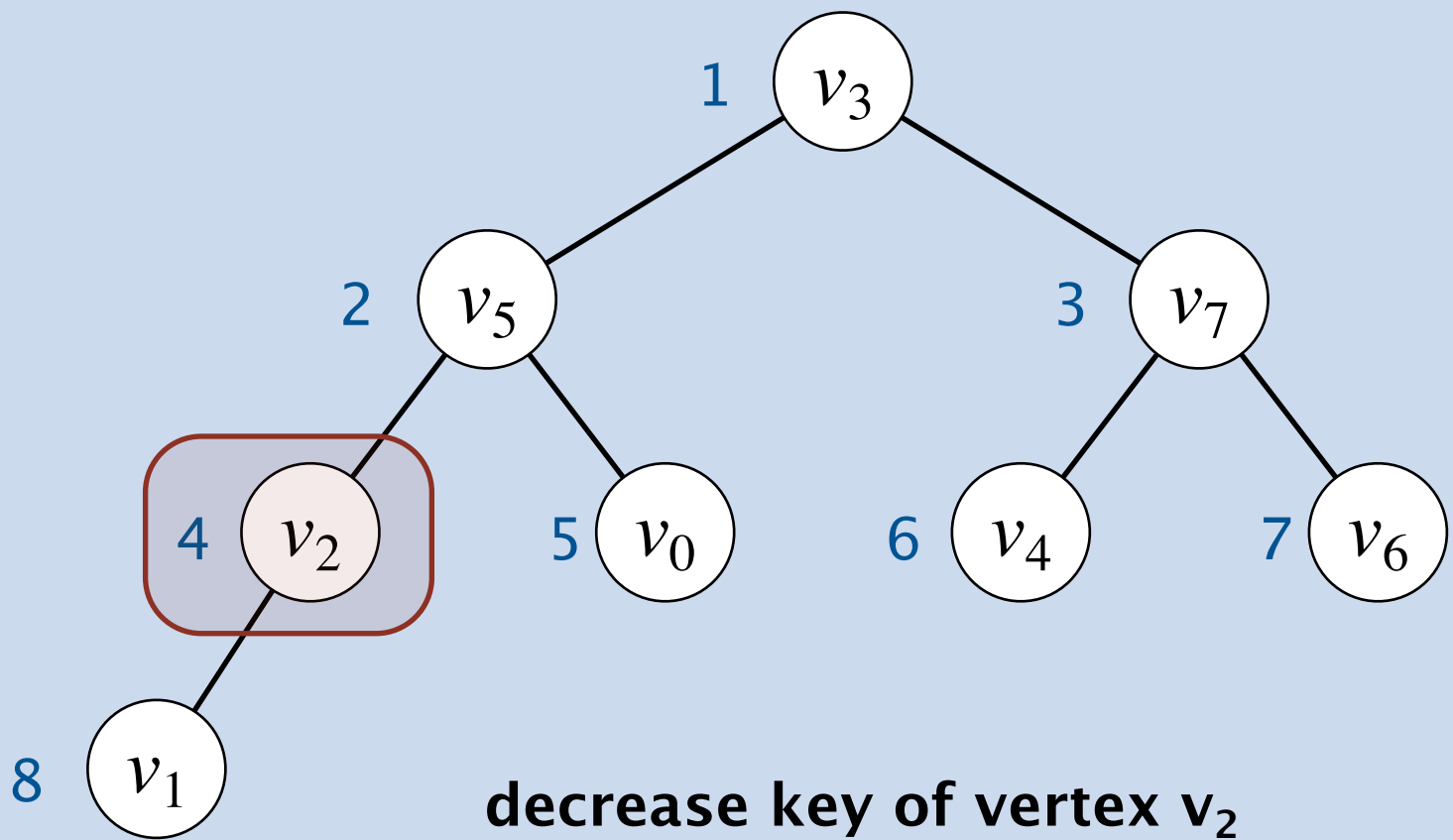
**Solution.**

- Find vertex in heap. How?
- Change priority of vertex and call `swim()` to restore heap invariant.

**Extra data structure.** Maintain an inverse array `qp[]` that maps from the vertex to the binary heap node index.

|                     | 0   | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     |
|---------------------|-----|-------|-------|-------|-------|-------|-------|-------|-------|
| <code>pq[]</code>   | —   | $v_3$ | $v_5$ | $v_7$ | $v_2$ | $v_0$ | $v_4$ | $v_6$ | $v_1$ |
| <code>qp[]</code>   | 5   | 8     | 4     | 1     | 6     | 2     | 7     | 3     | —     |
| <code>keys[]</code> | 1.0 | 2.0   | 3.0   | 0.0   | 6.0   | 8.0   | 4.0   | 2.0   | —     |

*vertex 2 has priority 3.0  
and is at heap index 4*



## Dijkstra's algorithm: which priority queue?

---

Number of PQ operations:  $V$  INSERT,  $V$  DELETE-MIN,  $\leq E$  DECREASE-KEY.

| PQ implementation | INSERT      | DELETE-MIN       | DECREASE-KEY | total            |
|-------------------|-------------|------------------|--------------|------------------|
| unordered array   | 1           | $V$              | 1            | $V^2$            |
| binary heap       | $\log V$    | $\log V$         | $\log V$     | $E \log V$       |
| d-way heap        | $\log_d V$  | $d \log_d V$     | $\log_d V$   | $E \log_{E/V} V$ |
| Fibonacci heap    | $1^\dagger$ | $\log V^\dagger$ | $1^\dagger$  | $E + V \log V$   |

$^\dagger$  amortized

### Bottom line.

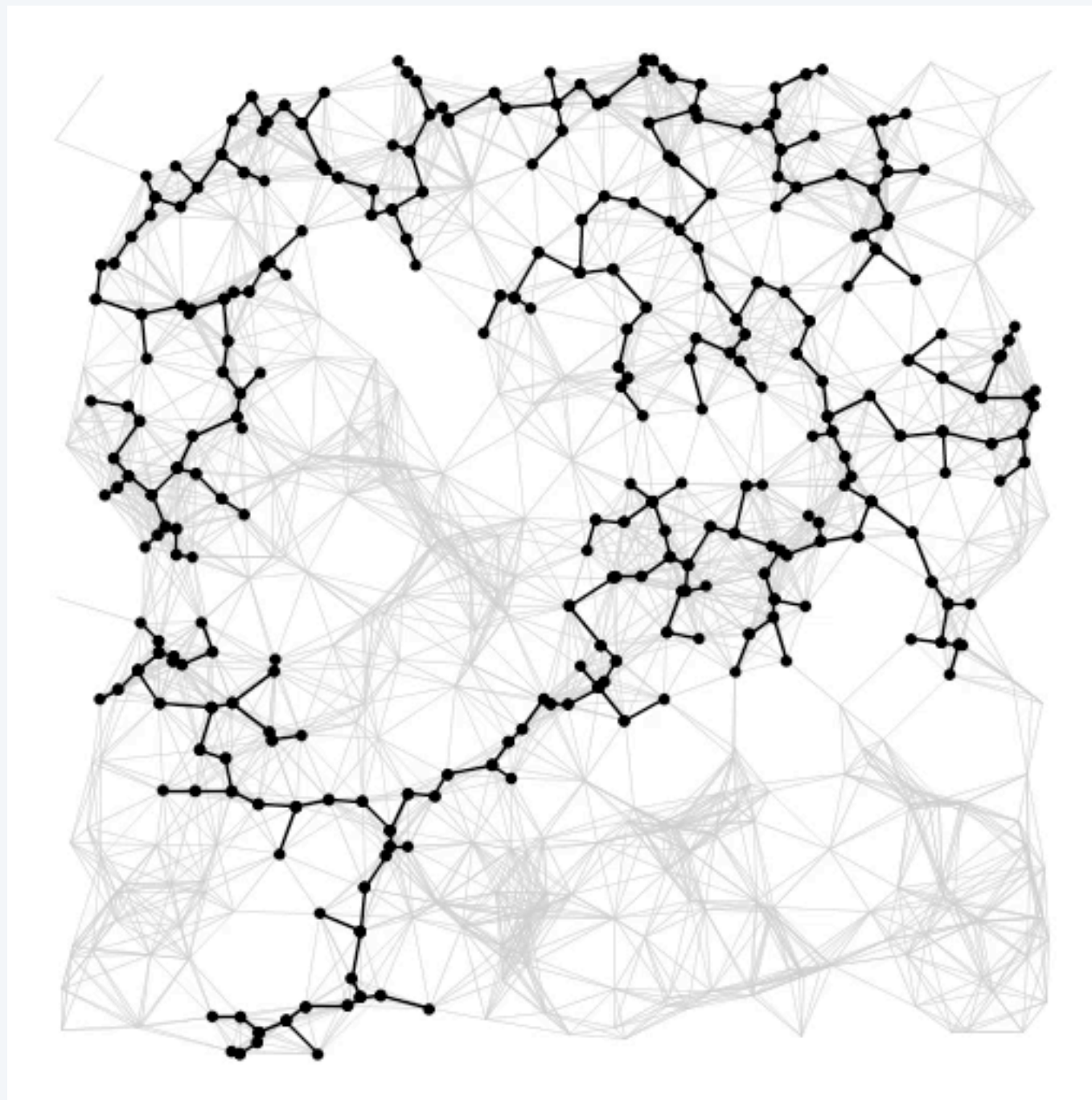
- Array implementation optimal for complete digraphs.
- Binary heap much faster for sparse digraphs.
- 4-way heap worth the trouble in performance-critical situations.
- Fibonacci heap best in theory, but probably not worth implementing.

# Priority-first search

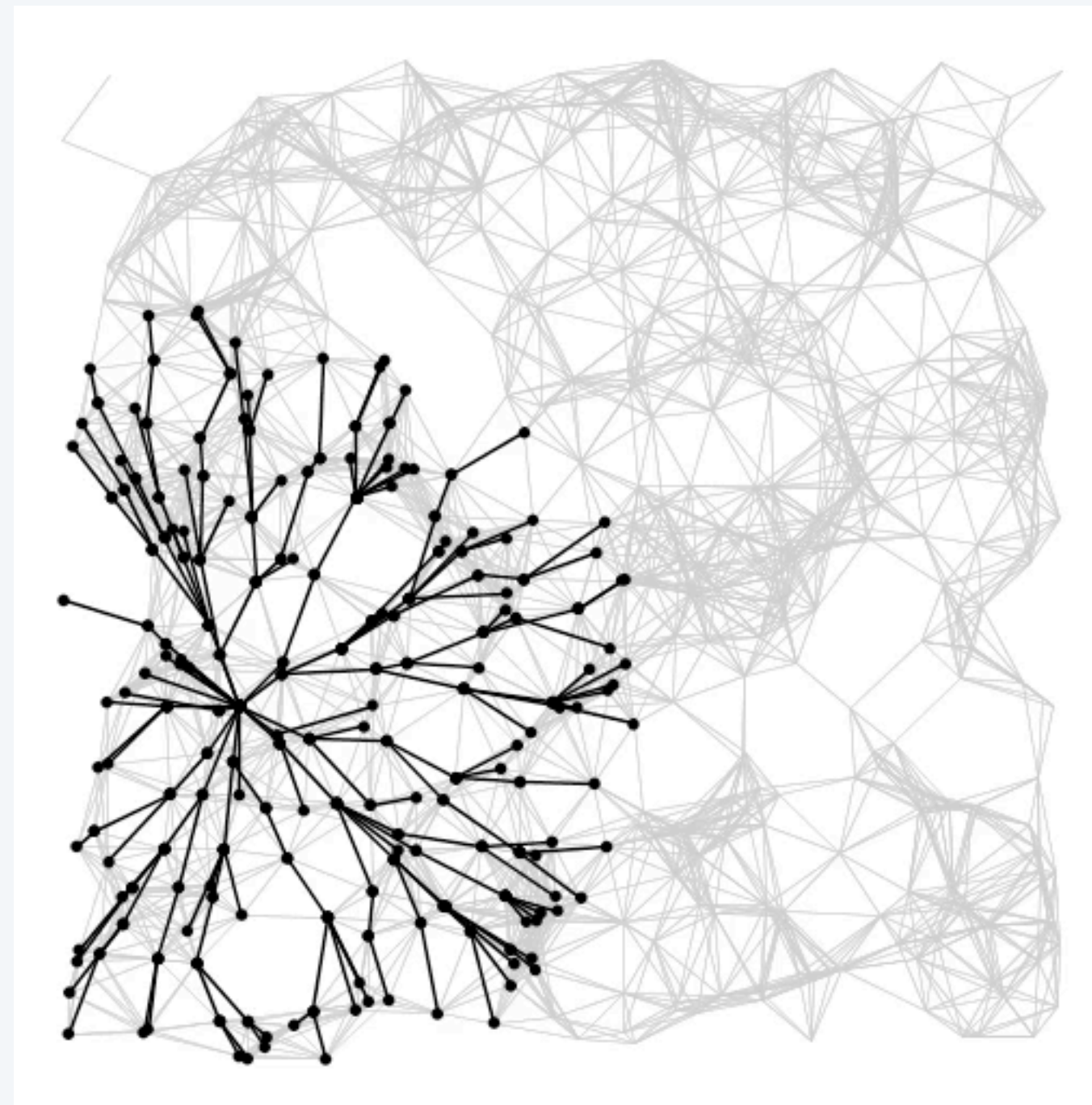
---

**Observation.** Prim and Dijkstra are essentially the same algorithm.

- Prim: Choose next vertex that is closest to **any vertex in the tree** (via an undirected edge).
- Dijkstra: Choose next vertex that is closest to the **source vertex** (via a directed path).



Prim's algorithm



Dijkstra's algorithm



# Algorithms for shortest paths

---

## Variations on a theme: vertex relaxations.

- Bellman–Ford: relax all vertices; repeat  $V - 1$  times.
- Dijkstra: relax vertices in order of distance from  $s$ .
- Topological sort: relax vertices in topological order. ← *see Section 4.4 and next lecture*

| algorithm        | worst-case running time | negative weights <sup>†</sup> | directed cycles |
|------------------|-------------------------|-------------------------------|-----------------|
| Bellman–Ford     | $E V$                   | ✓                             | ✓               |
| Dijkstra         | $E \log V$              |                               | ✓               |
| topological sort | $E$                     | ✓                             |                 |

<sup>†</sup> no negative cycles

# Which shortest paths algorithm to use?

---

Select algorithm based on properties of edge-weighted digraph.

- Negative weights (but no “negative cycles”): Bellman–Ford.
- Non-negative weights: Dijkstra.
- DAG: topological sort.

| algorithm        | worst-case running time | negative weights <sup>†</sup> | directed cycles |
|------------------|-------------------------|-------------------------------|-----------------|
| Bellman–Ford     | $E V$                   | ✓                             | ✓               |
| Dijkstra         | $E \log V$              |                               | ✓               |
| topological sort | $E$                     | ✓                             |                 |

<sup>†</sup> no negative cycles



# Credits

---

| image                         | source              | license                  |
|-------------------------------|---------------------|--------------------------|
| <i>Map of Princeton, N.J.</i> | <u>Google Maps</u>  |                          |
| <i>Broadway Tower</i>         | <u>Wikimedia</u>    | <u>CC BY 2.5</u>         |
| <i>Car GPS</i>                | <u>Adobe Stock</u>  | <u>education license</u> |
| <i>Queue for Registration</i> | <u>Noun Project</u> | <u>CC BY 3.0</u>         |
| <i>Dijkstra T-shirt</i>       | <u>Zazzle</u>       |                          |
| <i>Edsger Dijkstra</i>        | <u>Wikimedia</u>    | <u>CC BY-SA 3.0</u>      |

## A final thought

---

“ *Do only what only you can do.* ”

— Edsger W. Dijkstra

